

ROS 2 - Task Priority Framework

Alessio Caporali

alessio.caporali@unibo.it

DEI - LAR
University of Bologna
Viale del Risorgimento 2
40136 Bologna



Task Priority Framework

To employ the task priority framework with the Tiago robot, we will use three packages:

- **task priority**: *python package* that implements the task priority framework
- **lab4**: *ROS2 package* that contains
 - Node connecting the velocity controller of the Tiago with the task priority framework. Given a set of tasks, it computes the desired velocity of the robot
 - Client that send the desired end effector position to the node (above) for updating the end effector pose control task.
- **lab4_msgs**: *ROS2 package* that contains the srv used by the lab4 package

Environment Setup (step 1)

Update Tiago URDF files

To run the task priority framework, first we need to update the robot URDF description files.

Updated files are available on the virtuale page for the lab4 session. In particular the files are inside the *xacro* folder and they are:

```
arm.ros2_control.xacro  
arm.urdf.xacro  
wrist.ros2_control.xacro  
wrist.urdf.xacro
```

These files should replace those present inside the path:

```
~/tiago_ws/src/tiago_robot/tiago_description/urdf/arm/
```

Environment Setup (step 2)

Install Task Priority Framework Package

The task priority framework package is available on the virtuale page for the lab4 session.

It is a **simple python package** (not ROS), so it can be placed in any folder. However, for convenience, it can be placed inside the *tiago_ws* workspace.

Begin a python package, the easiest way to install it using pip. First we need to verify that we have pip installed. If not, we can install it using the command:

```
sudo apt install python3-pip
```

Then, to install the task priority framework package, just go inside its main directory (where you have the setup.py file) and execute the command:

```
pip install -e .
```

Now, you can import the **taks_priority** package anywhere in your virtual machine.

Environment Setup (step 3)

Install lab4 and lab4_msgs packages

The lab4 and lab4_msgs packages are available on the virtuale page for the lab4 session.

They should be placed inside the *tiago_ws* workspace.

To install the packages, just go inside the *tiago_ws* workspace and execute the command:

```
colcon build --packages-select lab4 lab4_msgs
```

Now, you can run the lab4 node and the client to update the end effector pose control task.

Environment Setup (step 4)

Activate Velocity Controller for the Tiago

To run the task priority framework, the velocity controller for the Tiago should be active.

To activate the velocity controller, one way is to modify the launch file that starts the Tiago simulation. The launch file is located inside the package *tiago_controller_configuration*:

```
~/tiago_ws/src/tiago_robot/tiago_controller_configuration/
```

The launch file is called *default_controllers.launch.py*.

Environment Setup (step 4)

Activate Velocity Controller for the Tiago

Configuration for the task priority framework:

```
#ld.add_action(arm_controller_launch) # JT CTRL  
ld.add_action(arm_controller_2_launch) # VELOCITY CTRL
```

Configuration for Moveit:

```
ld.add_action(arm_controller_launch) # JT CTRL  
#ld.add_action(arm_controller_2_launch) # VELOCITY CTRL
```

NOTE: Remember to rebuild the workspace after modifying the launch file.

NOTE2: Remember to change the launch file back to the original configuration when you want to use Moveit.

Environment Setup (step 5)

Check available controllers

```
ros2 control list_controllers
```

Among the provided output, you should have (look for the *arm velocity controller*):

```
student@ROS2-VirtualBox:~/tiago_ws$ ros2 control list_controllers
ft_sensor_controller[force_torque_sensor_broadcaster/ForceTorqueSensorBroadcaster] active
arm_velocity_controller[velocity_controllers/JointGroupVelocityController] active
mobile_base_controller[diff_drive_controller/DiffDriveController] active
head_controller [joint_trajectory_controller/JointTrajectoryController] active
joint_state_broadcaster[joint_state_broadcaster/JointStateBroadcaster] active
torso_controller [joint_trajectory_controller/JointTrajectoryController] active
gripper_controller [joint_trajectory_controller/JointTrajectoryController] active
```

documentation available at:

https://control.ros.org/humble/doc/ros2_control_demos/doc/index.html

TaskPriorityNode (inside lab4 package)

Below are the main steps to initialize the task priority framework:

```
# Init Task Priority Framework
self.robot = Robot(device='cpu')
self.end_link = 'arm_7_link'
self.chain_arm = 'arm'
self.robot.add_arm('tiago.urdf',
                  start_link='torso_lift_link',
                  ee_link=self.end_link,
                  task_control_name=self.chain_arm)

# Adding tasks
self.robot.add_task("JointLimitAvoidance", 0, self.chain_arm)
self.robot.add_task("EndEffectorControl", 1, self.chain_arm)
```

TaskPriorityNode (inside lab4 package)

Both tasks needs a set parameters and gains (or K) to be set.

```
# Setting task params
self.robot.set_task_param("EndEffectorControl",
                          self.chain_arm,
                          ee_target=np.array([.]),
                          control="POSITION",
                          K=[1.0, 1.0, 1.0, 1.5, 1.5, 1.5],
                          cycle_hz=100)

self.robot.set_task_param("JointLimitAvoidance",
                          self.chain_arm,
                          limit_offsets=np.array([0.03]*14),
                          gains=np.array([0.3]*14))
```

TaskPriorityNode (inside lab4 package)

The subscriber to the joint state will provide us both joint positions and velocities.

The publisher will send the desired velocity to the velocity controller.

```
# Subscriber to joint state
self.joints_sub = self.create_subscription(
    JointState,
    'joint_states',
    self.joint_state_callback)

# Velocity publisher
self.vel_pub = self.create_publisher(
    Float64MultiArray,
    '/arm_velocity_controller/commands')
```

Task Jacobian of Joint Limit Avoidance

$$\dot{x} = J\dot{q} \rightarrow \begin{bmatrix} \dot{x}_1^{\min} \\ \dot{x}_1^{\max} \\ \dot{x}_2^{\min} \\ \dot{x}_2^{\max} \\ \dot{x}_3^{\min} \\ \dot{x}_3^{\max} \\ \dot{x}_4^{\min} \\ \dot{x}_4^{\max} \\ \dot{x}_5^{\min} \\ \dot{x}_5^{\max} \\ \dot{x}_6^{\min} \\ \dot{x}_6^{\max} \\ \dot{x}_7^{\min} \\ \dot{x}_7^{\max} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \dot{q}_4 \\ \dot{q}_5 \\ \dot{q}_6 \\ \dot{q}_7 \end{bmatrix}$$

Running the TaskPriorityNode

First we need to start the gazebo simulation with the Tiago robot.

```
ros2 launch tiago_gazebo tiago_gazebo.launch.py
```

Then, we can start the task priority node:

```
ros2 run lab4 tiago_tasks_priority
```

By default, the node will start moving the end-effector target toward a default pose.

Sending New End-Effector Target

The client is used to send a new end effector target to the task priority node.

The client exploits the service **set_task_priority_ee_goal** provided by the task priority node. The service definition is inside *lab4_msgs*:

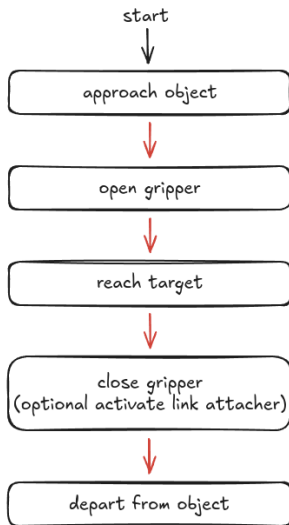
```
# EndEffectorGoal.srv
geometry_msgs/Point position
float64 roll
float64 pitch
float64 yaw
---
int64 result
```

The client accept a CLI input like the following (x, y, z, roll, pitch, yaw):

```
ros2 run lab4 tiago_tasks_priority_client
0.6 -0.5 0.0 3.14 0.0 0.785
```

Part 2: State Machine

State Machine



State Machine

Very basic approach:

```
# init state
self.state = 0

while taks_running:
    if self.state == 0:
        // do something and wait for the task to be completed
        self.state = 1

    elif self.state == 1:
        // do something and wait for the task to be completed
        self.state = 2

    ...
```

Optional: use more advanced libraries for state machines (e.g., *pytransitions*)